2026-10-25 MySQL表的内外连接

- 1. MySQL 表的内外连接
  - 1.1 为什么需要表连接
    - 1.1.1 数据通常分散在多张表中
    - 1.1.2 笛卡尔积是理解连接的基础
    - 1.1.3 连接的本质
  - 1.2 三种连接的核心区别
- 2. 内连接：只保留匹配成功的数据
  - 2.1 内连接的概念
    - 2.1.1 内连接就是“笛卡尔积 + 连接条件”
    - 2.1.2 显式内连接语法
  - 2.2 内连接案例：显示 SMITH 的姓名和部门名称
    - 2.2.1 使用逗号和 WHERE
    - 2.2.2 使用 INNER JOIN
  - 2.3 推荐写法：连接条件放 ON，普通过滤放 WHERE
  - 2.4 内连接的易错点
    - 2.4.1 漏写连接条件
    - 2.4.2 多表同名字段要标明表名
- 3. 左外连接：完整保留左表
  - 3.1 为什么需要外连接
  - 3.2 左外连接的概念
  - 3.3 左外连接语法
  - 3.4 案例：查询所有学生及其成绩
    - 3.4.1 建表与准备数据
    - 3.4.2 查询需求
    - 3.4.3 结果分析
    - 3.4.4 用右连接改写
- 4. 右外连接：完整保留右表
  - 4.1 右外连接的概念
  - 4.2 右外连接语法
  - 4.3 案例：显示所有成绩
    - 4.3.1 查询需求
    - 4.3.2 结果分析
  - 4.4 左右外连接的等价关系
  - 4.5 方向错了会怎样
    - 4.5.1 先明确到底要保留哪一侧

        - 4.5.2 一对多关系会增加结果行数
- 5. 综合案例：部门与员工信息
    - 5.1 数据背景
    - 5.2 两种写法
    - 5.3 更清晰的综合查询
        - 5.3.1 预期结果
    - 5.4 本案例的判断方法
- 6. SQL 编写方法与易错点
    - 6.1 推荐的书写顺序
    - 6.2 选择连接类型的方法
        - 6.2.1 只要匹配成功的数据
        - 6.2.2 左表必须完整保留
        - 6.2.3 右表必须完整保留
    - 6.3 常见错误
        - 6.3.1 把“有匹配”误认为“保留全部”
        - 6.3.2 外连接方向写反
        - 6.3.3 认为 NULL 表示数据库中真的存了 NULL
        - 6.3.4 忘记表别名
- 7. 练习与总结
    - 7.1 课件练习
        - 7.1.1 LeetCode: Rank Scores
        - 7.1.2 LeetCode: Exchange Seats
    - 7.2 本课必须掌握的三条结论
        - 7.2.1 内连接只保留匹配成功的数据
        - 7.2.2 左外连接完整保留左表
        - 7.2.3 右外连接完整保留右表
    - 7.3 一页速查

1. MySQL 表的内外连接

1.1 为什么需要表连接

1.1.1 数据通常分散在多张表中

关系型数据库为了减少重复数据，会把不同含义的数据拆成不同的表。例如：

- `emp` 员工表保存员工信息和部门编号。

- `dept` 部门表保存部门编号和部门名称。

员工姓名在 `emp` 中，部门名称在 `dept` 中。如果业务需要“员工姓名 + 所属部门名称”，只查询一张表无法得到完整信息，就需要把两张表连接起来。

1.1.2 笛卡尔积是理解连接的基础

如果直接把两张表放在一起查询：

```
SELECT *
FROM emp, dept;
```

MySQL 会把第一张表的每一行与第二张表的每一行逐一组合，得到两表行数的乘积，这个结果叫作**笛卡尔积**。

大部分组合并没有业务意义。例如某位员工属于 20 号部门，却被组合到了 10 号、30 号部门。为了得到真实对应关系，必须增加连接条件：

```
SELECT *
FROM emp, dept
WHERE emp.deptno = dept.deptno;
```

1.1.3 连接的本质

连接概括为：

1. 先把多张表按照连接条件组合成一张临时结果表。
2. 再对组合后的结果做筛选、投影和排序。

因此，理解连接时始终要分清两件事：

- **连接条件**：决定两张表的行如何对应，例如 `emp.deptno = dept.deptno`。
- **过滤条件**：决定组合后的结果中哪些行需要保留，例如 `ename = 'SMITH'`。

1.2 三种连接的核心区别

连接方式	核心语义	左表未匹配	右表未匹配
`INNER JOIN` 内连接	只保留左右两边都能匹配的组合	丢弃	丢弃
`LEFT JOIN` 左外连接	完整保留左表，再拼接右表中匹配的数据	保留，右表列为 `NULL`	看左表是否有匹配；无匹配则不会单独出现
`RIGHT JOIN` 右外连接	完整保留右表，再拼接左表中匹配的数据	看右表是否有匹配；无匹配则不会单独出现	保留，左表列为 `NULL`

`LEFT OUTER JOIN` 与 `LEFT JOIN` 等价，`RIGHT OUTER JOIN` 与 `RIGHT JOIN` 等价。日常 SQL 中通常省略 `OUTER`。

2. 内连接：只保留匹配成功的数据

2.1 内连接的概念

2.1.1 内连接就是“笛卡尔积 + 连接条件”

> 内连接实际上就是利用 `WHERE` 子句对两张表形成的笛卡尔积进行筛选。前面学习的查询都是内连接，也是在开发过程中使用最多的连接查询。

这句话需要从两个角度理解：

- 用逗号把多张表放进 `FROM`，再用 `WHERE` 指定两表的编号相等，本质上就是内连接。
- 使用 `INNER JOIN ... ON ...` 只是语法表达更加清楚，并不是另一种完全不同的连接。

2.1.2 显式内连接语法

基本语法：

```
SELECT 字段
FROM 表1
INNER JOIN 表2
    ON 连接条件
    AND 其他条件;
```

也可以省略 `INNER`：

```
SELECT 字段
FROM 表1
JOIN 表2
    ON 连接条件;
```

多张表连接时，可以继续在后面追加 `JOIN ... ON ...`：

```
SELECT ...
FROM 表1
JOIN 表2 ON 表1.id = 表2.id
JOIN 表3 ON 表2.id = 表3.id;
```

2.2 内连接案例：显示 SMITH 的姓名和部门名称

2.2.1 使用逗号和 WHERE

第一种写法：

```
SELECT ename, dname
FROM EMP, DEPT
WHERE EMP.deptno = DEPT.deptno
  AND ename = 'SMITH';
```

执行逻辑可以分成两步：

1. `EMP` 和 `DEPT` 先形成笛卡尔积。
2. `WHERE` 保留员工部门编号等于部门编号，并且员工姓名为 `SMITH` 的记录。

2.2.2 使用 INNER JOIN

第二种写法：

```
SELECT ename, dname
FROM EMP
INNER JOIN DEPT
        ON EMP.deptno = DEPT.deptno
       AND ename = 'SMITH';
```

两种写法在本例中得到的结果相同。`INNER JOIN` 把“连接两张表”和“筛选员工姓名”放在同一段 SQL 中表达，结构更直观。

2.3 推荐写法：连接条件放 ON，普通过滤放 WHERE

为了区分“表的连接方式”和“业务筛选条件”，实际开发中更推荐：

```
SELECT e.ename, d.dname
FROM emp AS e
JOIN dept AS d
  ON e.deptno = d.deptno
WHERE e.ename = 'SMITH';
```

这样一眼就能看出：

- `ON e.deptno = d.deptno` 是两张表如何对应。
- `WHERE e.ename = 'SMITH'` 是最终查询需要哪个员工。

当涉及多张表或复杂业务条件时，这种分阶段写法比把所有条件都塞进 `WHERE` 更容易阅读。

2.4 内连接的易错点

2.4.1 漏写连接条件

如果只写 `FROM emp JOIN dept` 而不写 `ON`，或者写了一个永远成立的连接条件，就可能得到规模很大的笛卡尔积。

2.4.2 多表同名字段要标明表名

两张表都有 `deptno` 时，应写成：

```
emp.deptno = dept.deptno
```

使用表别名后可以写：

```
e.deptno = d.deptno
```

这既避免了字段歧义，也让 SQL 更简洁。

3. 左外连接：完整保留左表

3.1 为什么需要外连接

内连接只保留两边都能匹配的数据，但真实业务中经常需要保留“没有对应记录”的主体。例如：

- 有些学生没有参加考试，但仍要显示这些学生。
- 有些部门暂时没有员工，但仍要显示这些部门。

如果使用内连接，这些没有匹配项的主体会被直接丢弃。外连接就是为解决这个问题而设计的。

3.2 左外连接的概念

如果要求**左表完整显示**，就使用左外连接：

- 左表的每一行都必须保留。
- 右表能够匹配时，就拼接右表数据。
- 右表没有匹配时，右表对应的列用 `NULL` 补齐。

定义：

> 如果联合查询时，左表要完全显示，我们就说是左外连接。

3.3 左外连接语法

```
SELECT 字段
FROM 表1
LEFT JOIN 表2
    ON 连接条件;
```

其中 `表1` 是左表，`表2` 是右表。左表不会因为右表没有匹配数据而消失。

3.4 案例：查询所有学生及其成绩

3.4.1 建表与准备数据

使用两张简单的示意表。

学生表：

```sql
CREATE TABLE stu (
    id INT,
    name VARCHAR(30)
);

INSERT INTO stu
VALUES
    (1, 'jack'),
    (2, 'tom'),
    (3, 'kity'),
    (4, 'nono');
```

成绩表：

```sql
CREATE TABLE exam (
    id INT,
    grade INT
);

INSERT INTO exam
VALUES
    (1, 56),
    (2, 76),
    (11, 8);
```

观察数据可以发现：

- 学生表中有 1、2、3、4 号学生。
- 成绩表中只有 1、2、11 号记录。
- 3、4 号学生没有成绩。
- 11 号成绩没有对应学生。

3.4.2 查询需求

查询所有学生的成绩；如果某个学生没有成绩，也要显示学生信息。

题目要求保留“所有学生”，因此必须以学生表 `stu` 作为左表。

```
SELECT *
FROM stu
LEFT JOIN exam
       ON stu.id = exam.id;
```

3.4.3 结果分析

预期结果可以表示为：

stu.id	stu.name	exam.id	exam.grade
1	jack	1	56
2	tom	2	76
3	kity	NULL	NULL
4	nono	NULL	NULL

3、4 号学生没有被成绩表匹配，但因为是左表数据，所以仍然保留；成绩相关列用 `NULL` 表示“没有成绩”。

如果把 `LEFT JOIN` 换成 `INNER JOIN`，结果只会剩下一号和二号学生，因为只有他们有成绩。

3.4.4 用右连接改写

把两张表交换顺序后，也可以得到相同效果：

```
SELECT *
FROM exam
RIGHT JOIN stu
        ON stu.id = exam.id;
```

这里仍以 `stu` 的数据完整保留为目标，只不过 `stu` 被放到了右表的位置。

4. 右外连接：完整保留右表

4.1 右外连接的概念

如果要求**右表完整显示**，就使用右外连接：

- 右表的每一行都必须保留。
- 左表能够匹配时，就拼接左表数据。
- 左表没有匹配时，左表对应的列用 `NULL` 补齐。

> 如果联合查询时，右表要完全显示，我们就说是右外连接。

4.2 右外连接语法

```
SELECT 字段
FROM 表1
RIGHT JOIN 表2
        ON 连接条件;
```

这里的 `表2` 是必须完整保留的右表。

4.3 案例：显示所有成绩

4.3.1 查询需求

对 `stu` 和 `exam` 做联合查询，显示所有成绩；即使某条成绩没有对应学生，也要显示出来。

题目要求保留所有成绩，因此以成绩表 `exam` 作为右表：

```
SELECT *
FROM stu
RIGHT JOIN exam
        ON stu.id = exam.id;
```

4.3.2 结果分析

预期结果可以表示为：

stu.id	stu.name	exam.id	exam.grade
1	jack	1	56
2	tom	2	76
NULL	NULL	11	8

11 号成绩没有对应学生，但因为成绩表是右表，所以这条成绩仍然保留，学生相关列用 `NULL` 补齐。

4.4 左右外连接的等价关系

单纯从功能上看，`RIGHT JOIN` 并不是必须存在的新能力。交换两张表的位置并改用 `LEFT JOIN`，通常可以得到等价结果：

```sql
-- 保留 stu：按左连接写
SELECT *
FROM stu
LEFT JOIN exam
       ON stu.id = exam.id;

-- 保留 stu：也可以交换顺序，按右连接写
SELECT *
FROM exam
RIGHT JOIN stu
        ON stu.id = exam.id;
```

实际选择哪一种写法，更多是围绕“读者能否一眼看出哪张表需要完整保留”来决定。

4.5 方向错了会怎样

4.5.1 先明确到底要保留哪一侧

如果需求是“显示所有学生”，却只从成绩表出发写成：

```sql
SELECT *
FROM exam
```

```
LEFT JOIN stu
       ON stu.id = exam.id;
```

由于这里完整保留的是 `exam`，3、4 号没有成绩的学生仍然不会出现，需求并没有满足。要保留所有学生，应改成：

```
SELECT *
FROM stu
LEFT JOIN exam
       ON stu.id = exam.id;
```

反过来，如果需求是“保留所有成绩”，从 `exam` 出发的 `LEFT JOIN` 就是正确的。因此写外连接前应先回答一个问题：

> **哪张表即使没有匹配数据，也必须完整出现在结果中？**

4.5.2 一对多关系会增加结果行数

左表一行可能匹配右表多行。例如一个部门有多名员工，执行“部门左连接员工”后，该部门会出现多行，每行对应一名员工。这是正常的连接结果，不是重复错误。

5. 综合案例：部门与员工信息

5.1 数据背景

课堂上使用了经典的 `scott` 数据库：

- `emp` 员工表中出现过的部门号有 10、20、30。
- `dept` 部门表中有 10、20、30、40 号部门。
- 40 号部门暂时没有员工。

如果要同时显示部门名称和员工信息，还要保留没有员工的部门，就必须以 `dept` 表为主表。

5.2 两种写法

以部门为左表：

```sql
SELECT d.dname, e.*
FROM dept AS d
LEFT JOIN emp AS e
       ON d.deptno = e.deptno;
```

或者交换顺序，以部门为右表：

```sql
SELECT d.dname, e.*
FROM emp AS e
RIGHT JOIN dept AS d
        ON d.deptno = e.deptno;
```

这两种写法都表达了同一件事：**完整保留部门表，再用员工表补充员工信息。**

5.3 更清晰的综合查询

为了让输出包含部门编号和员工姓名，并按部门号排序，可以写成：

```sql
SELECT
    d.deptno,
    d.dname,
    e.ename
FROM dept AS d
LEFT JOIN emp AS e
       ON d.deptno = e.deptno
ORDER BY d.deptno ASC;
```

5.3.1 预期结果

- 10、20、30 号部门对应各自员工。
- 有多名员工的部门会出现多行。
- 40 号部门没有员工，但仍然保留，`ename` 为 `NULL`。

排序后能够更直观地看到：

```
10 号部门 → 有多名员工
20 号部门 → 有多名员工
```

```
30 号部门 → 有多名员工
40 号部门 → 没有员工，员工相关字段为 NULL
```

5.4 本案例的判断方法

看到“显示所有部门”“没有员工的部门也要显示”这类描述时，可以按下面的顺序分析：

1. 确定要输出的信息来自哪些表：部门名称来自 `dept`，员工信息来自 `emp`。
2. 确定哪张表必须完整保留：题目要求所有部门，所以保留 `dept`。
3. 把 `dept` 放在左表，使用 `LEFT JOIN`；或者把 `dept` 放在右表，使用 `RIGHT JOIN`。
4. 连接条件写成 `dept.deptno = emp.deptno`。
5. 最后根据输出要求选择字段并排序。

6. SQL 编写方法与易错点

6.1 推荐的书写顺序

多表查询可以按阶段组织：

```
SELECT 需要的字段
FROM 左表 AS 别名
连接类型 JOIN 右表 AS 别名
    ON 两表之间的对应条件
WHERE 普通业务筛选条件
ORDER BY 排序字段;
```

这种阅读顺序与课堂强调的分析方法一致：先确定表怎样连接，再确定最终要筛选什么。

6.2 选择连接类型的方法

6.2.1 只要匹配成功的数据

使用 `INNER JOIN`。

例如：只查询有成绩的学生。

6.2.2 左表必须完整保留

使用 `LEFT JOIN`，并把不能丢数据的表放在左边。

例如：所有学生及其成绩、所有部门及其员工。

6.2.3 右表必须完整保留

使用 `RIGHT JOIN`，并把不能丢数据的表放在右边。

例如：所有成绩及对应学生。

6.3 常见错误

6.3.1 把“有匹配”误认为“保留全部”

内连接不会保留未匹配行。需要保留某一侧全部数据时，必须使用外连接。

6.3.2 外连接方向写反

`LEFT JOIN` 中，左表无论是否匹配都会保留；右表则未必。不要把左右连接当成单纯的排版差异。

6.3.3 认为 `NULL` 表示数据库中真的存了 `NULL`

外连接结果里的 `NULL` 可能只表示“这次连接没有找到匹配行”，不一定意味着原表中原本就存了 `NULL`。

6.3.4 忘记表别名

连接多张表时，推荐使用短别名：

```
FROM emp AS e
JOIN dept AS d ON e.deptno = d.deptno
```

这样字段来源清楚，SQL 也更短。

7. 练习与总结

7.1 课件练习

7.1.1 LeetCode: Rank Scores

题目：`rank-scores`

链接：https://leetcode.cn/problems/rank-scores/

练习重点：多表查询、结果排序，以及 SQL 中排名问题的表达方式。

7.1.2 LeetCode: Exchange Seats

题目：`exchange-seats`

链接：https://leetcode.cn/problems/exchange-seats/

练习重点：相邻记录交换、条件分支，以及结果集组织。

7.2 本课必须掌握的三条结论

7.2.1 内连接只保留匹配成功的数据

`INNER JOIN ... ON ...` 与“逗号连接表 + `WHERE` 连接条件”在语义上等价；显式 `JOIN` 更适合复杂查询。

7.2.2 左外连接完整保留左表

左表每一行都会保留；右表无匹配时，右表列补 `NULL`。

7.2.3 右外连接完整保留右表

右表每一行都会保留；左表无匹配时，左表列补 `NULL`。交换表顺序并改用左连接，通常可以得到同样的结果。

7.3 一页速查

```sql
-- 1. 内连接：两边都匹配
SELECT *
FROM A
INNER JOIN B
        ON A.id = B.id;

-- 2. 左外连接：完整保留 A
SELECT *
FROM A
LEFT JOIN B
       ON A.id = B.id;

-- 3. 右外连接：完整保留 B
SELECT *
FROM A
RIGHT JOIN B
        ON A.id = B.id;

-- 4. 左连接改写右连接：交换表顺序
SELECT *
FROM B
LEFT JOIN A
       ON A.id = B.id;
```